

DA2303

TEAM 11

DISK-ANN ALGORITHM IMPROVEMENTS

Ramana Gopal
Sree Hasini
Chiru Thejaswi
Mithilesh

VAMANA INDEXING AND GREEDY SEARCH IMPLEMENTATION

Phase 1: The Build (Construction)

- Initialization: Start with a random graph and select a random node as the fixed entry point.
- Pass 1: Connects points to their nearest neighbors (local accuracy).
- Pass 2: Uses a higher (alpha) threshold to add long-distance "shortcuts" across the graph.
- Robust Pruning: Ensures a point doesn't have redundant neighbors. It only keeps a neighbor if that neighbor provides a unique direction not already covered by others.

Parameter	What it is
R(Degree)	Max neighbors per node.
L(Build List)	Search depth during build.
Alpha	Pruning threshold.

Phase 2: The Search (Querying)

- Start: Begin at a random node.
- Greedy Traversal: Move to the neighbor closest to the query vector.
- Search Buffer: Maintain a set of "best-seen" candidates. The algorithm explores the neighbors of these candidates iteratively.
- Termination: Stop once the closest nodes in the buffer can no longer be improved.

Parameter	What it is
L(Search List)	Size of the "best-seen" list.
Gamma	Degree multiplier triggering neighbor pruning

IMPLEMENTED APPROACHES

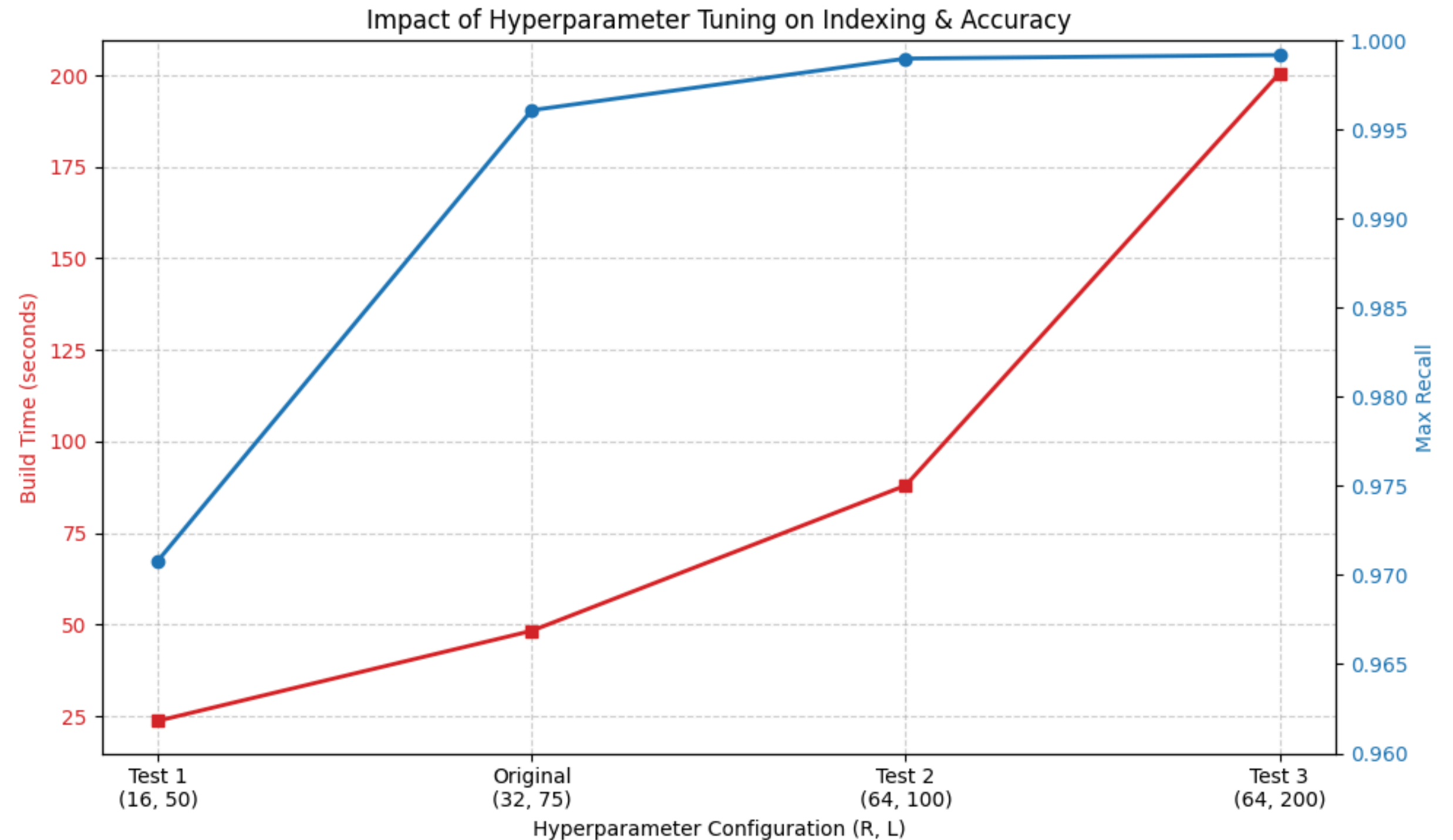
- **Hyper parameter tuning**
- **Medoid-based start-node initialization**
- **Noise reduction**
- **Concurrent search optimization**
- **Two-Pass index construction**
- **Adaptive beam-width search(L)**
- **Multistage seed(K- mediods) + Early-Stop**
- **Double-Pass + Medoid + Adaptive Beam**

HYPERPARAMETER TUNING

- Original (R=32, L=75)

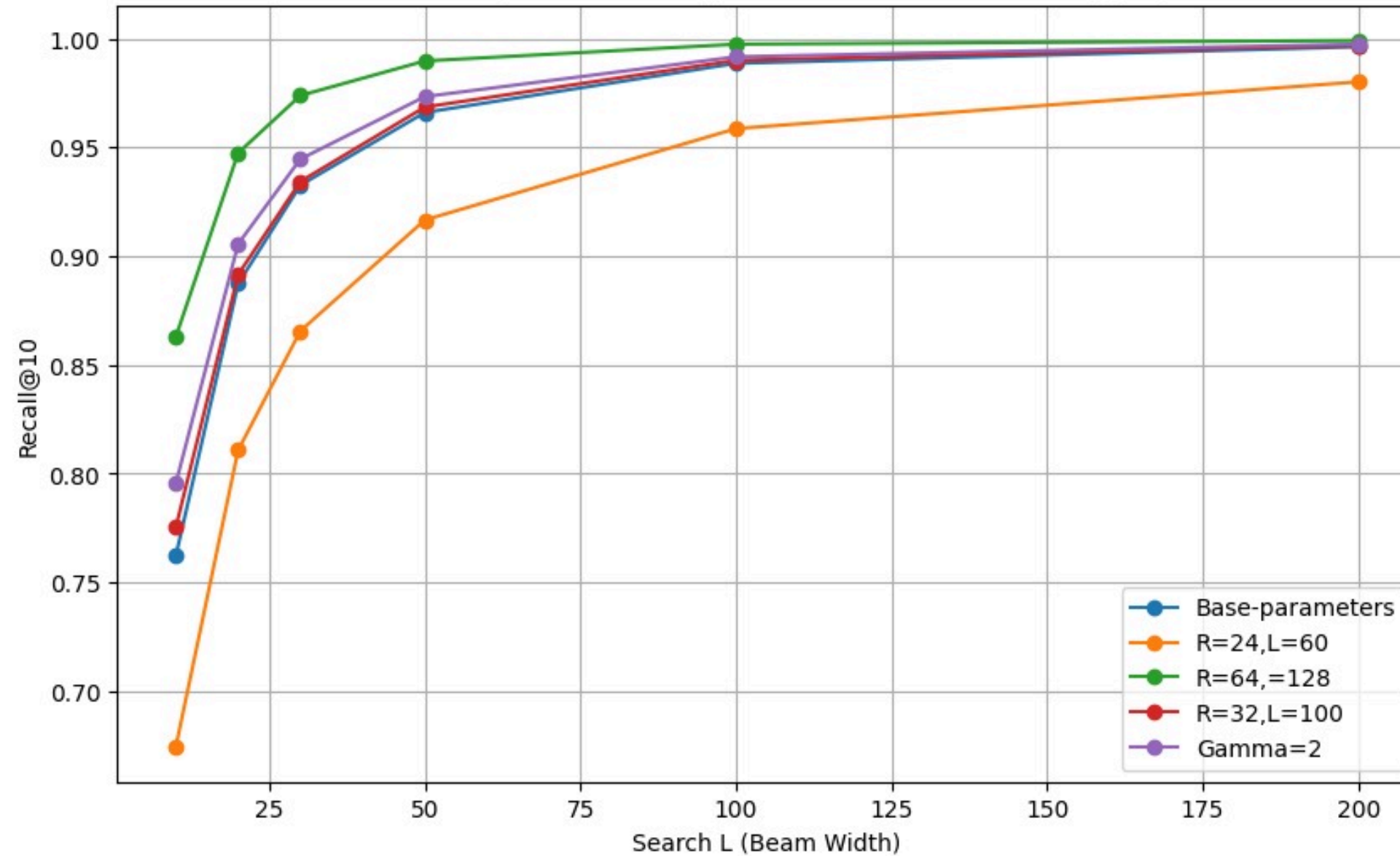
Variations Implemented

- Sparse (R=16, L=50)
- Standard Dense (R=64, L=100)
- High-Quality Dense (R=64, L=200)

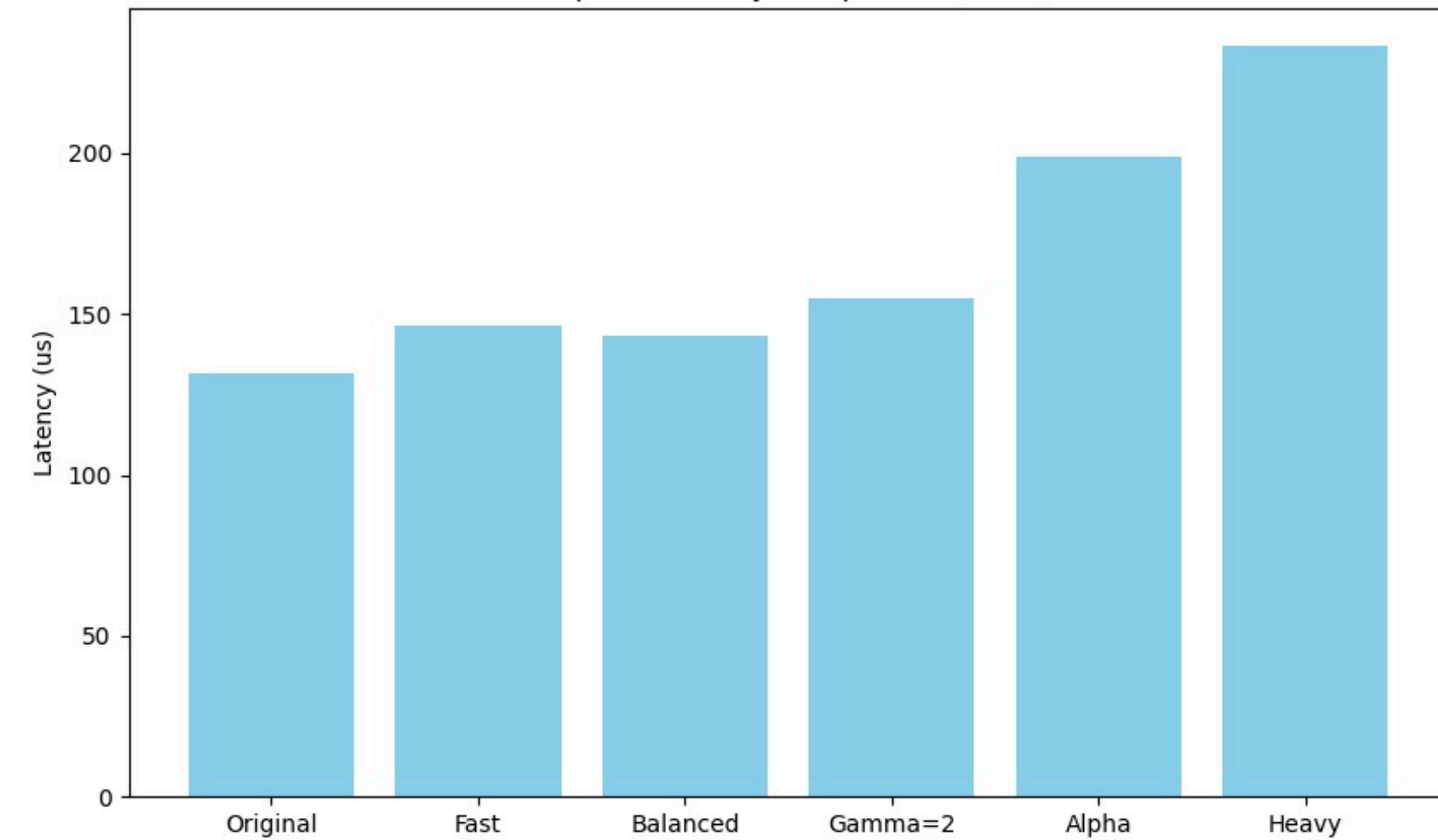


HYPERPARAMETER TUNING

Graph 1: Recall Comparison for original model with different hyperparameters



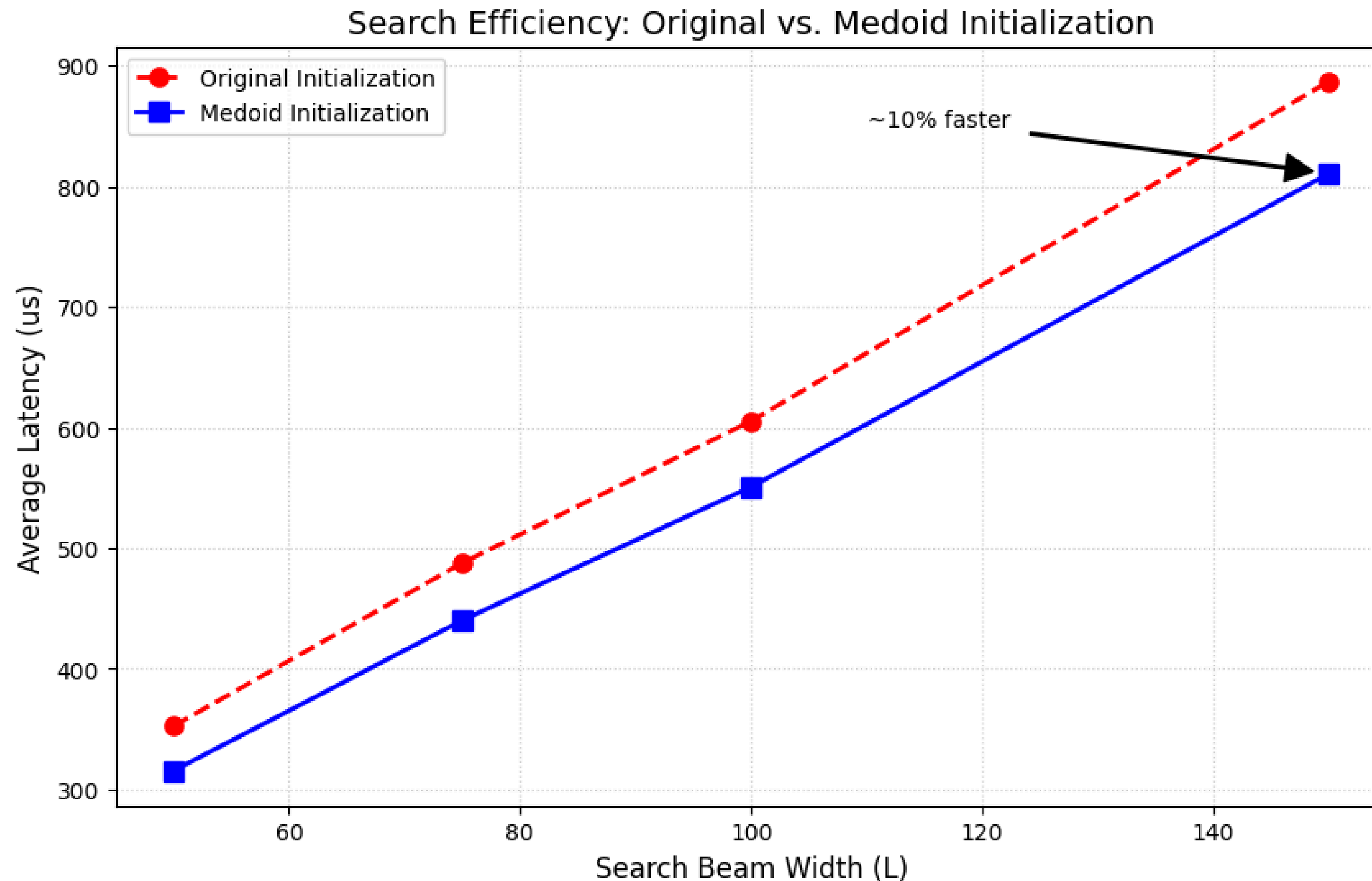
Graph 2: Latency Comparison (L=10)



MEDIOD INITIALISATION

Replace arbitrary entry points with a Medoid (the point closest to the dataset's centroid).

This minimises "long-distance hops," strictly reducing latency and distance computations while preserving full recall.



NOISE REDUCTION METHOD

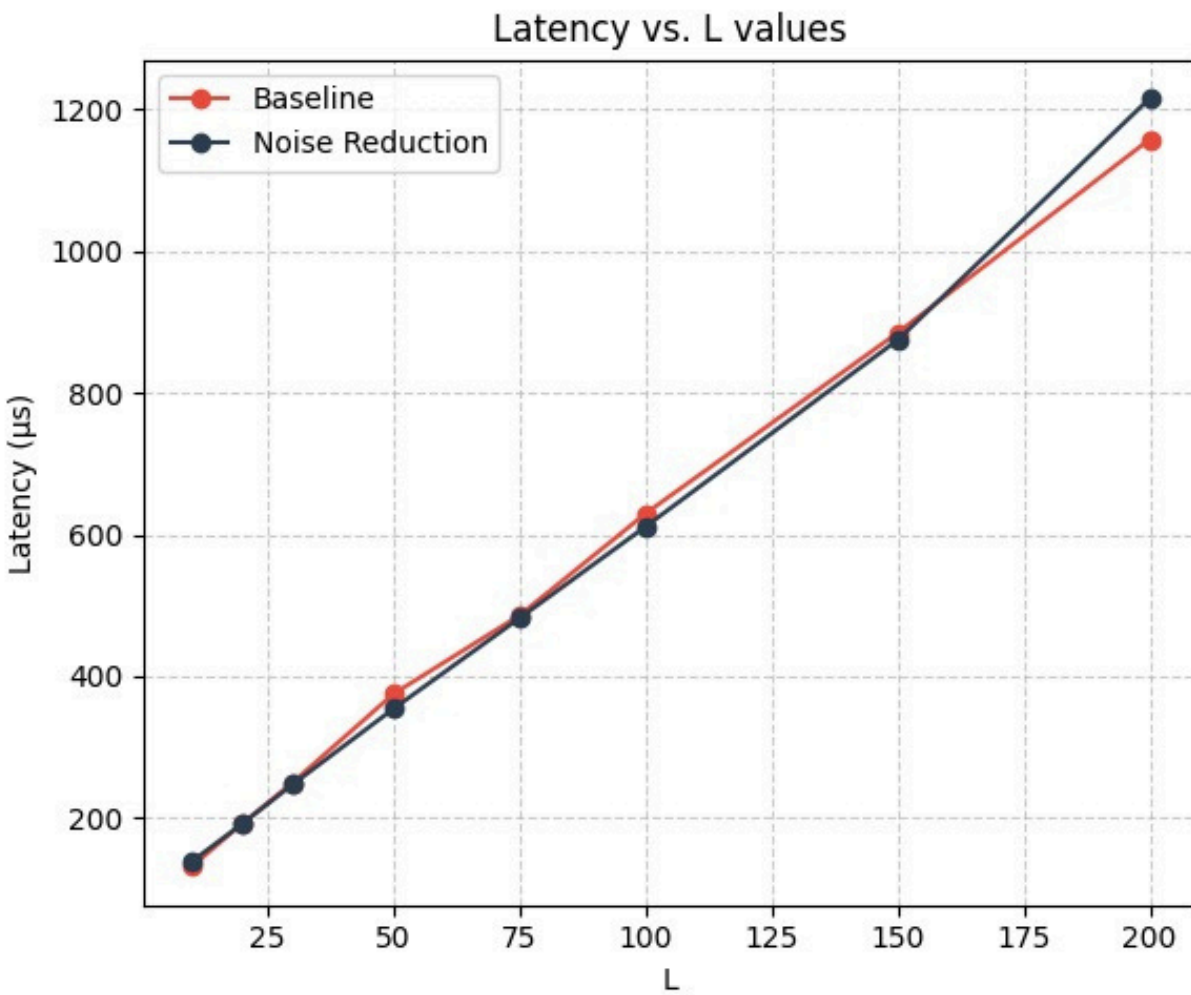
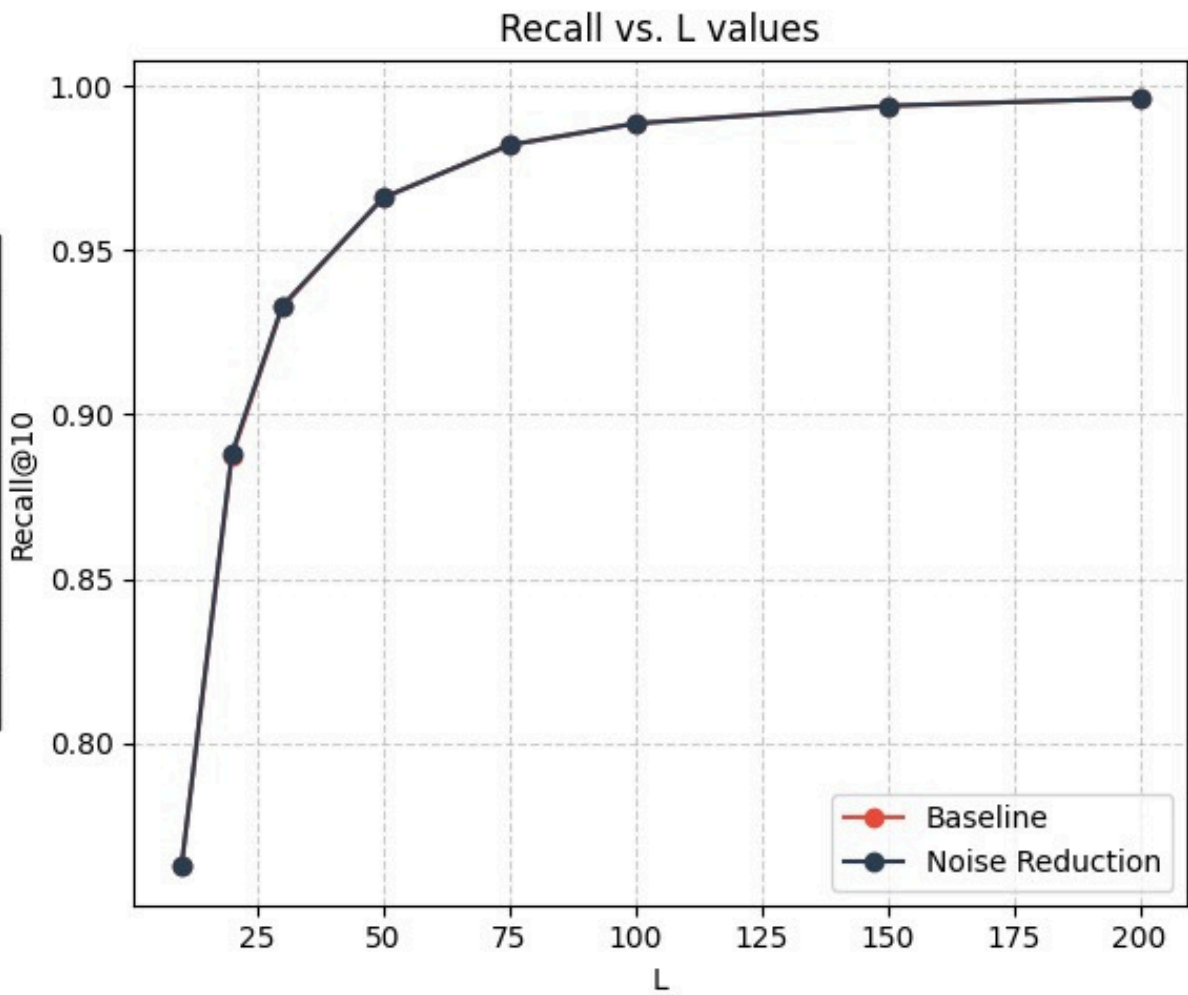
Noise reduction slightly smooths the vectors, but it doesn't change the relative ordering of distances, so nearest neighbors remain almost the same

There is very little “noise” to remove, as we are using SIFT1M data set which has very little noise

Plot 3: Baseline vs. Noise Reduction Optimization

Tabular Comparison

L	Recall (Base)	Recall (Noise)	Lat us (Base)	Lat us (Noise)
10.0	0.7624	0.7625	131.4	138.5
20.0	0.8873	0.8882	192.0	191.6
30.0	0.9326	0.9331	249.9	247.2
50.0	0.966	0.9659	375.0	354.0
75.0	0.9819	0.982	485.8	481.9
100.0	0.9886	0.9885	629.6	611.1
150.0	0.9939	0.9939	884.7	874.5
200.0	0.9962	0.9962	1157.8	1215.6

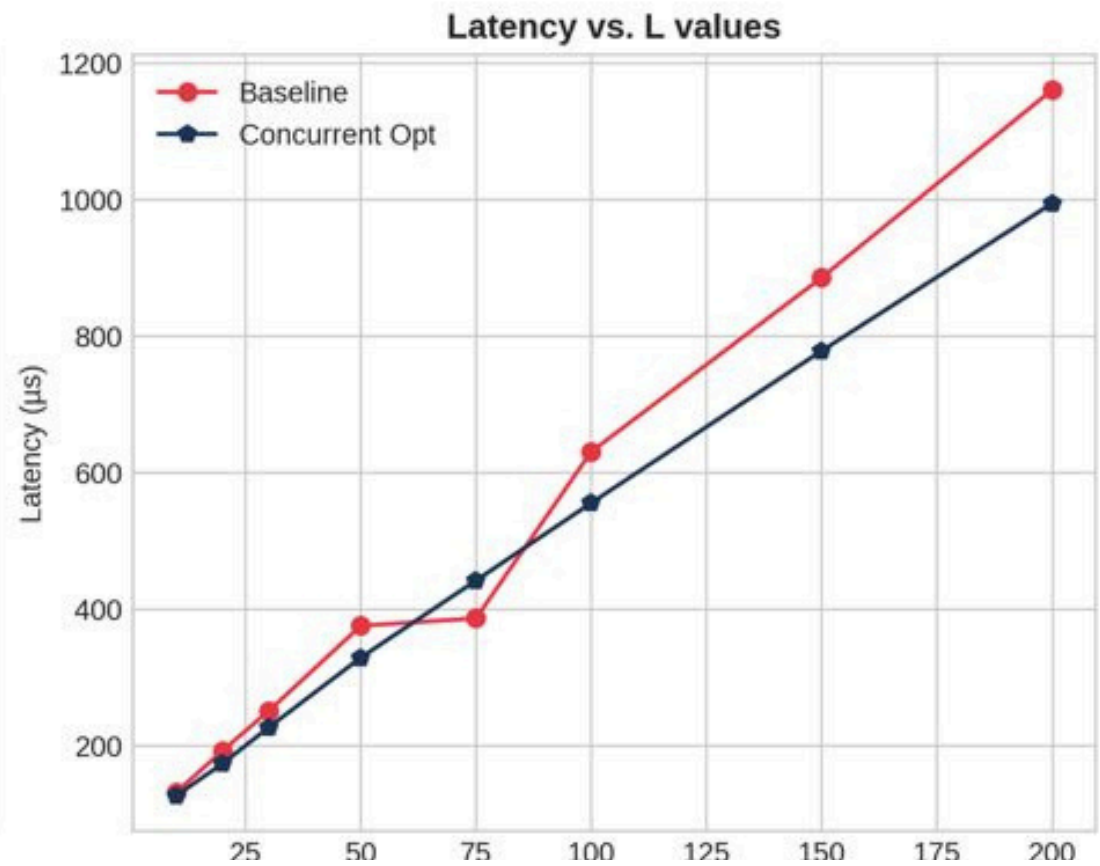
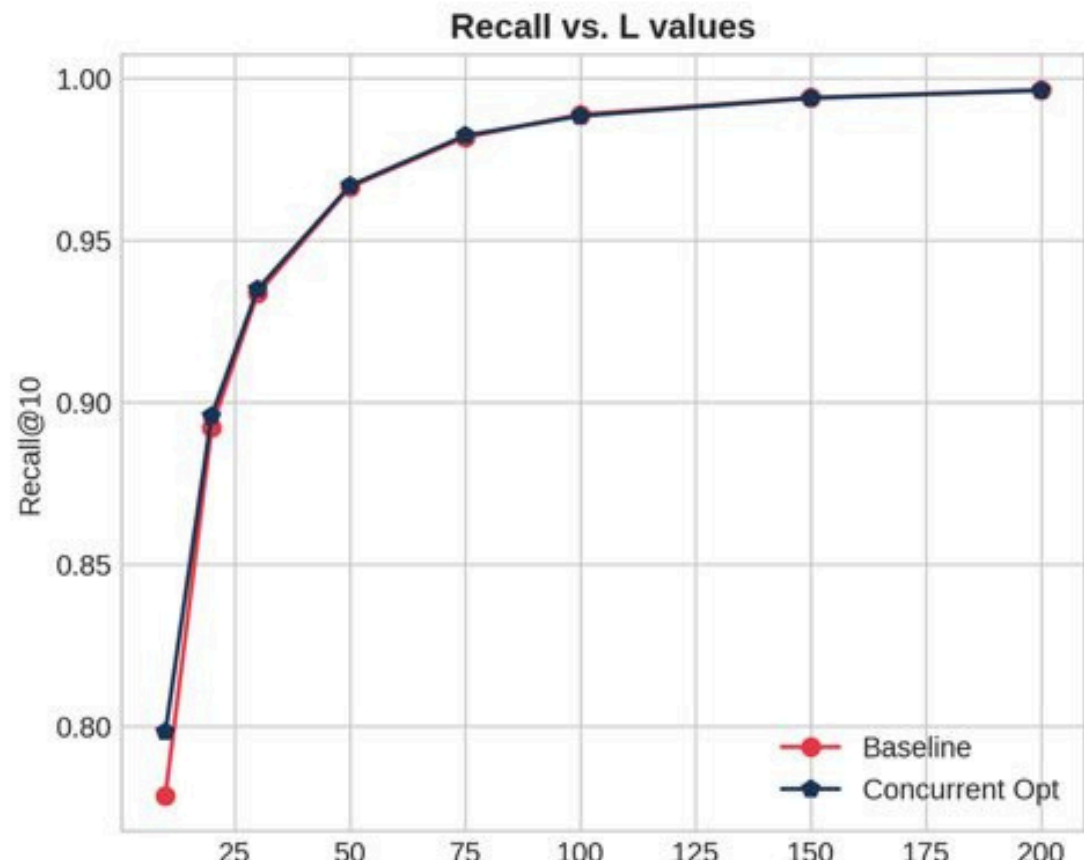


CONCURRENT OPTIMIZATION

Optimize performance by replacing temporary memory allocations with a permanent, pre-allocated workspace to handle multiple searches faster and more efficiently.

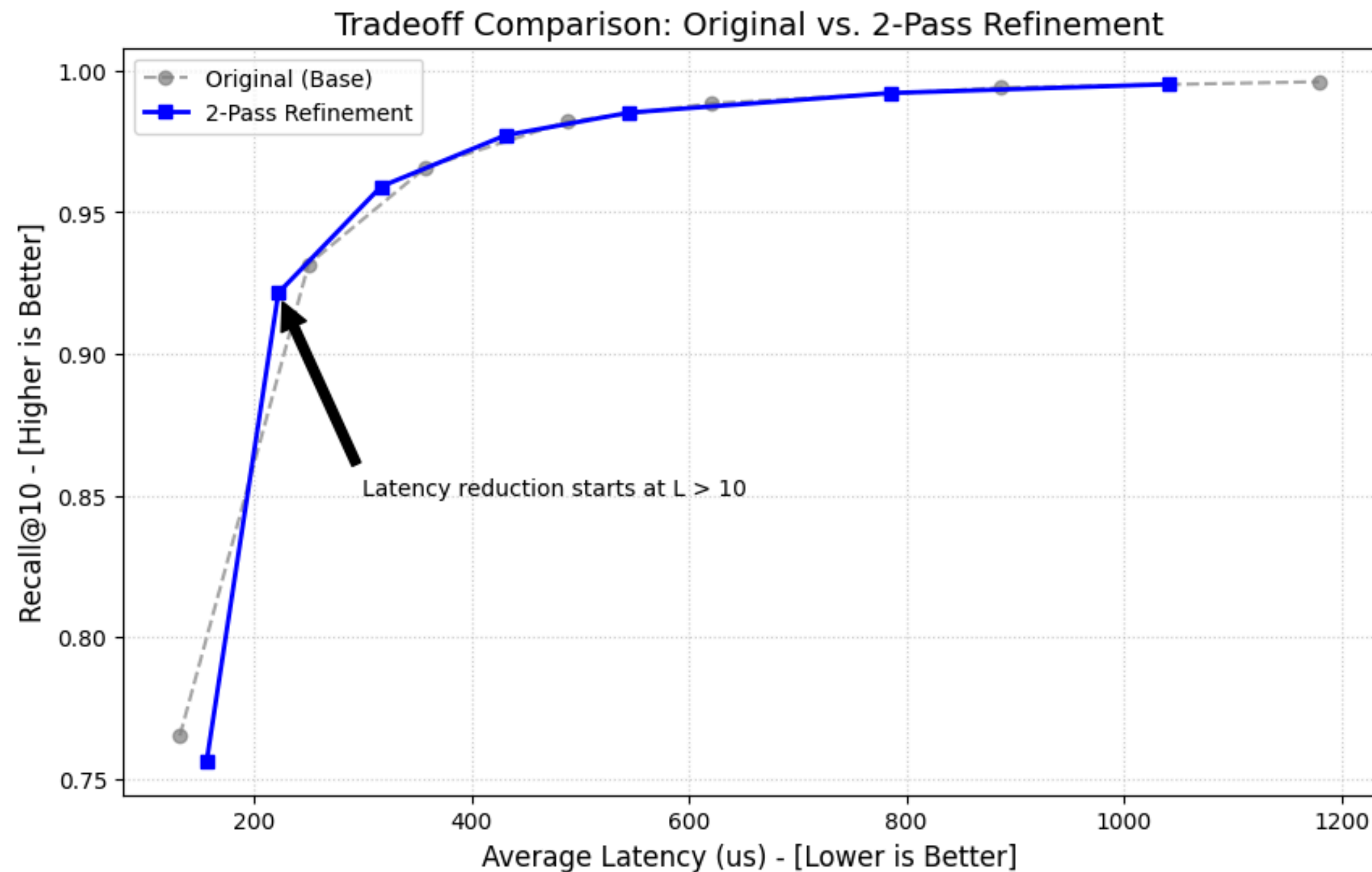
Plot 3: Baseline vs. Concurrent Optimization

Tabular Comparison				
L	Recall (Base)	Recall (Conc)	Lat μ s (Base)	Lat μ s (Conc)
10.0	0.7785	0.7981	131.0	126.3
20.0	0.8921	0.8957	192.0	173.0
30.0	0.9335	0.9348	250.0	226.0
50.0	0.9663	0.9667	375.0	328.3
75.0	0.9817	0.9822	386.0	441.3
100.0	0.9887	0.9883	630.0	554.9
150.0	0.9939	0.9938	885.0	777.8
200.0	0.9963	0.9962	1160.0	993.8

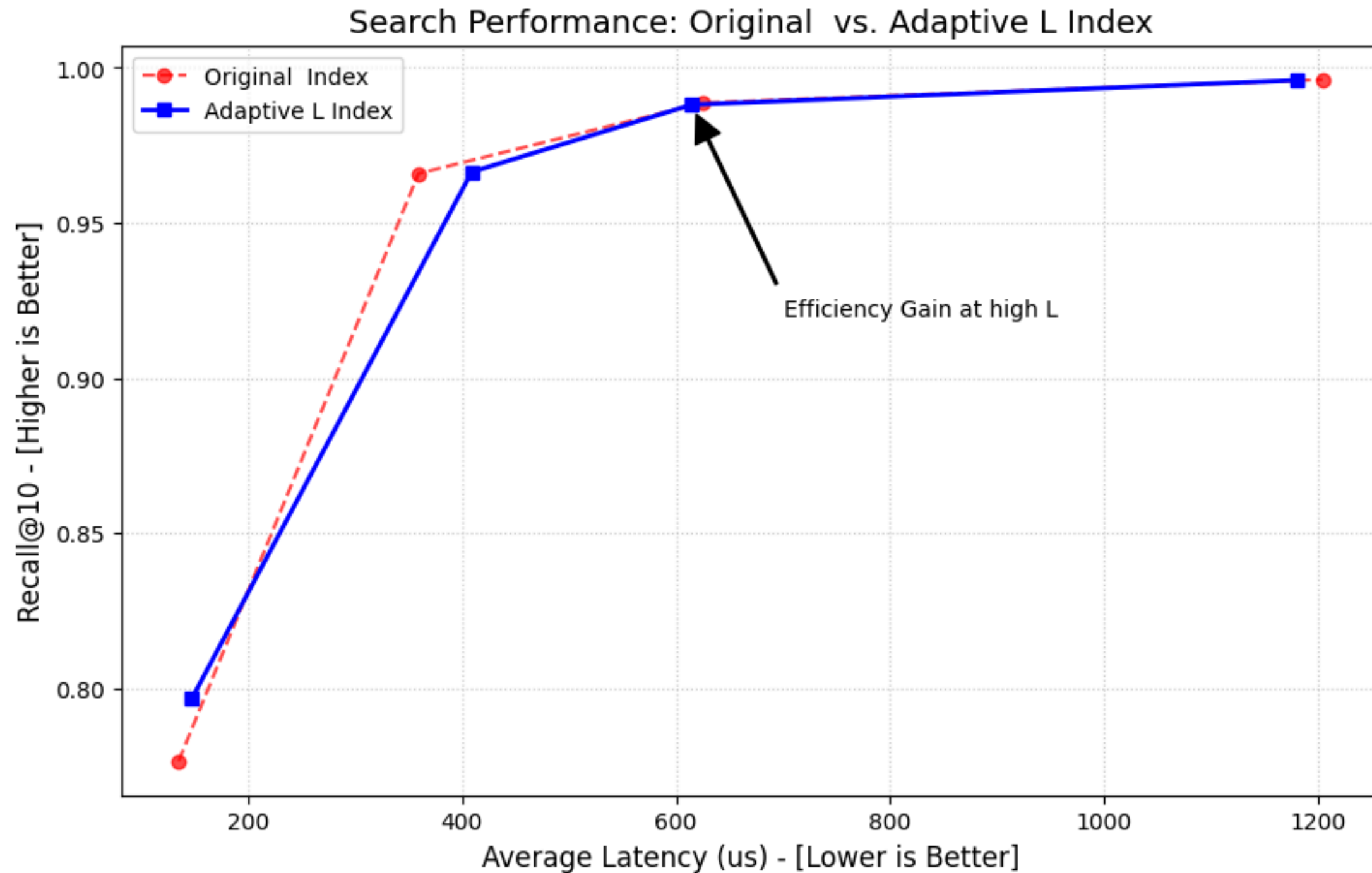


TWO-PASS BUILD

- Pass 1: Builds a sparse initial graph to establish highly localised connectivity.
- Pass 2 (Refinement): Re-inserts all points by relaxing the threshold to introduce long-range edges to speed up routing.



ADAPTIVE BEAM WIDTH(L)



- **Dynamic L:** Replaced the fixed beam width with a scaling current L.
- **L_{max}:** Introduced a cap to define the maximum allowable search depth.

MULTISTAGE SEED(K- MEDIODS) + EARLY-STOP

◆ Build Stage (Construction) — Multistage Seed + Early Stop

- **Seed Selection:** Use K-medoids to pick well-spread data points as starting anchors (instead of random node).
- **Stage 1 (Coarse):** Connect nodes using far/global medoid-based neighbors.
- **Stage 2 (Refinement):** Improve connections using closer/local neighbors.
- **Robust Pruning:** Keep only useful neighbors (remove redundant ones).
- **Early Stop (Build):** Stop adding neighbors when no significant improvement in distances.

**WE REACH THE RIGHT REGION
FASTER, DO LESS WORK, AND STILL
FIND GOOD NEIGHBORS.**

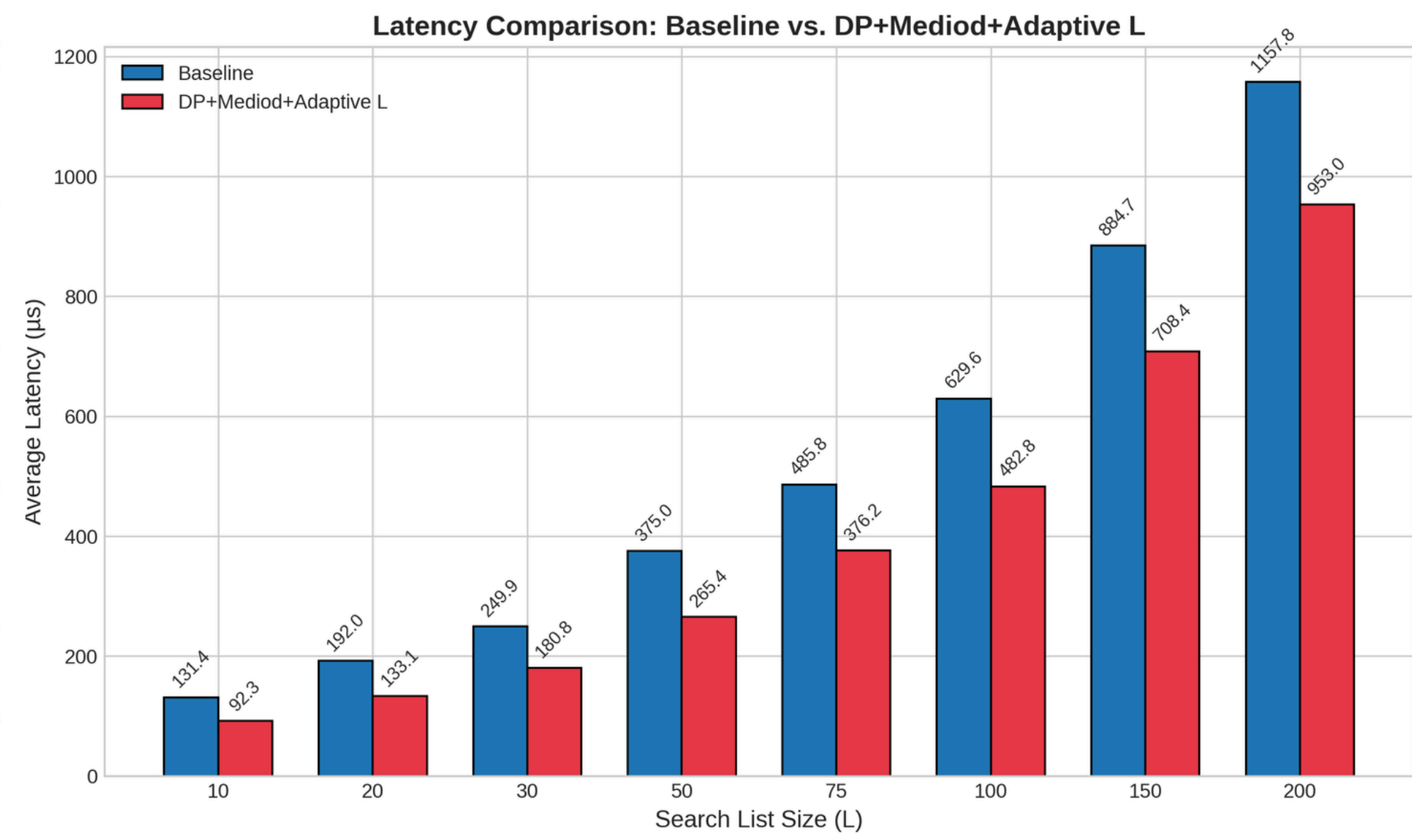
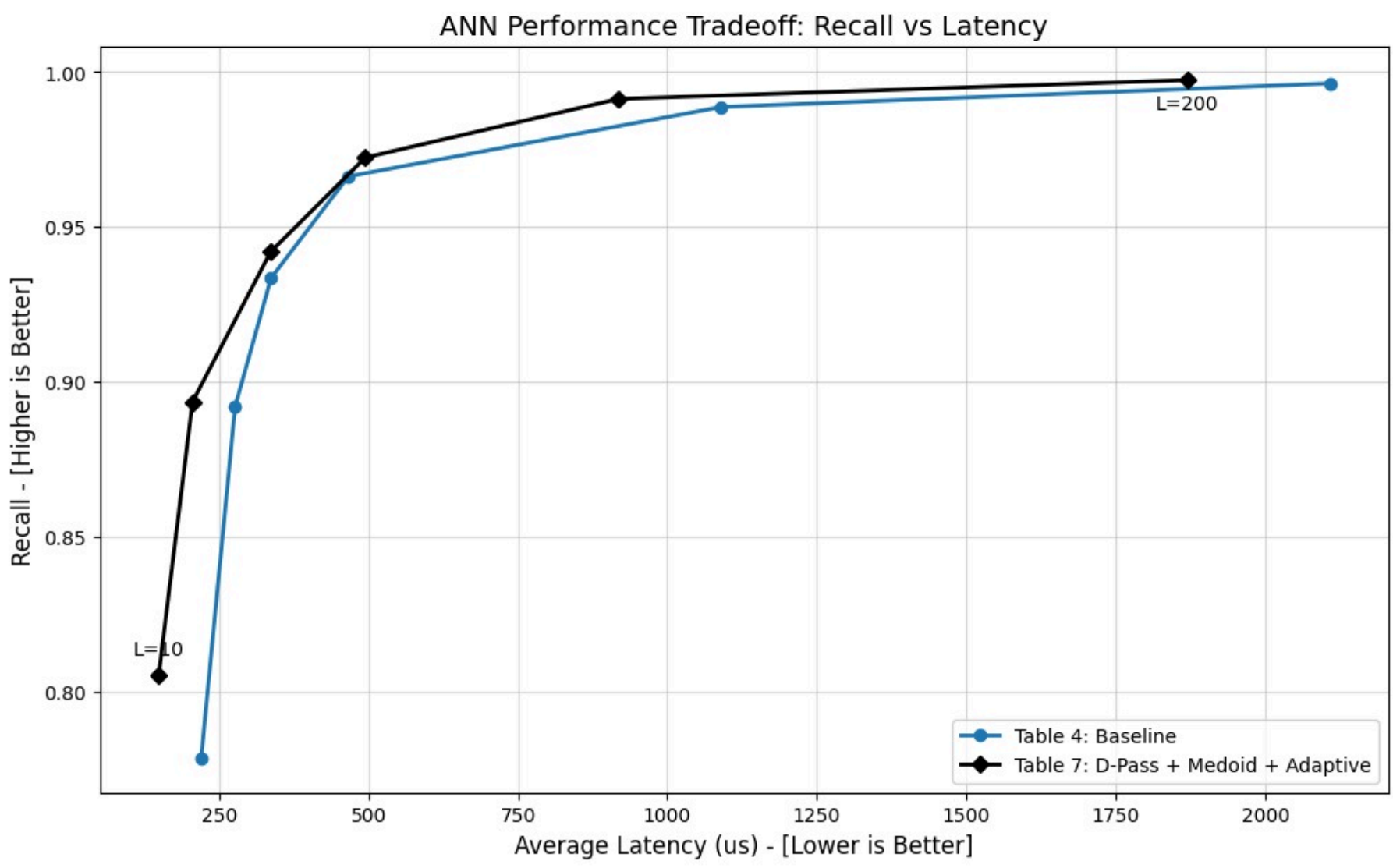
◆ Search Stage (Querying) — Multistage Seed + Early Stop

- **Start:** Begin from closest medoid (not random entry point).
- **Stage-wise Search:**
 - First jump through global nodes
 - Then move to local neighbors
- **Greedy Traversal:** Always move toward nodes closer to the query.
- **Search Buffer (L):** Maintain best candidate nodes and explore them.
- **Early Stop (Search):** Stop when no better neighbors are found in the buffer.

◆ Final intuition

Metric	Effect
Latency	↓ Faster
Recall	≈ Same / ↑ Slightly
Avg Distance	↓ Better

ENSEMBLING : DOUBLE-PASS + MEDOID + ADAPTIVE L



DOUBLE PASS VS DOUBLE PASS + MEDOID

Plot 2: Double Pass vs. Double Pass + Medoid Initialization

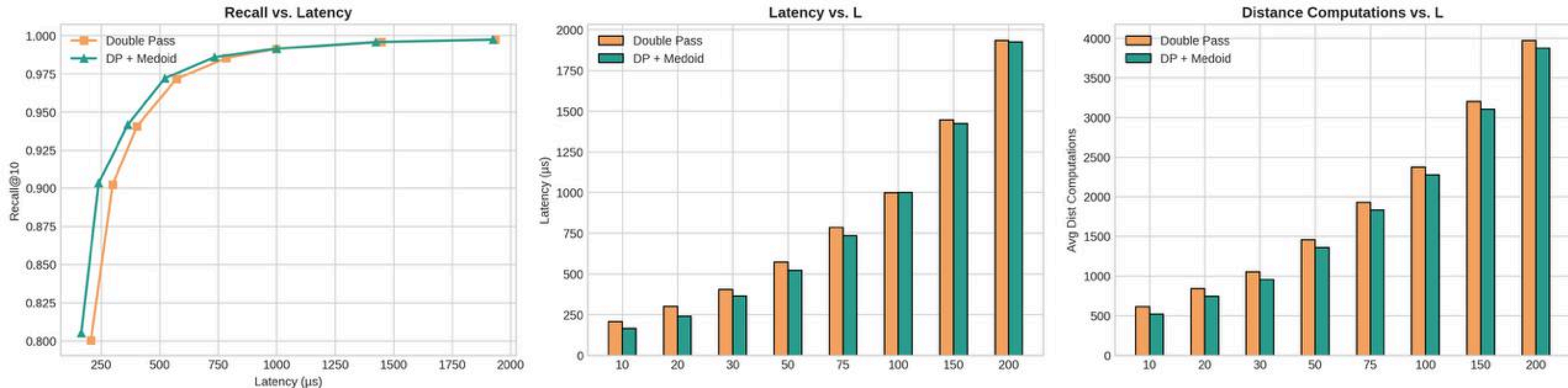


Table 1: Improvement: (Double Pass) vs. (Medoid + Double Pass)

Metric	L=50 (Old → New)	L=100 (Old → New)	L=200 (Old → New)
Avg Latency	571.7 → 521.8 (+8.7%)	997.8 → 999.2 (~0%)	1934.2 → 1925.0 (+0.5%)
Dist Cmps	1453.7 → 1357.6 (+6.6%)	2371.1 → 2274.5 (+4.1%)	3972.0 → 3874.2 (+2.5%)
Recall@10	0.9716 → 0.9720	0.9912 → 0.9914	0.9973 → 0.9973

DOUBLE PASS +MEDOID+ADAPTIVE L VS DOUBLE PASS + MEDOID

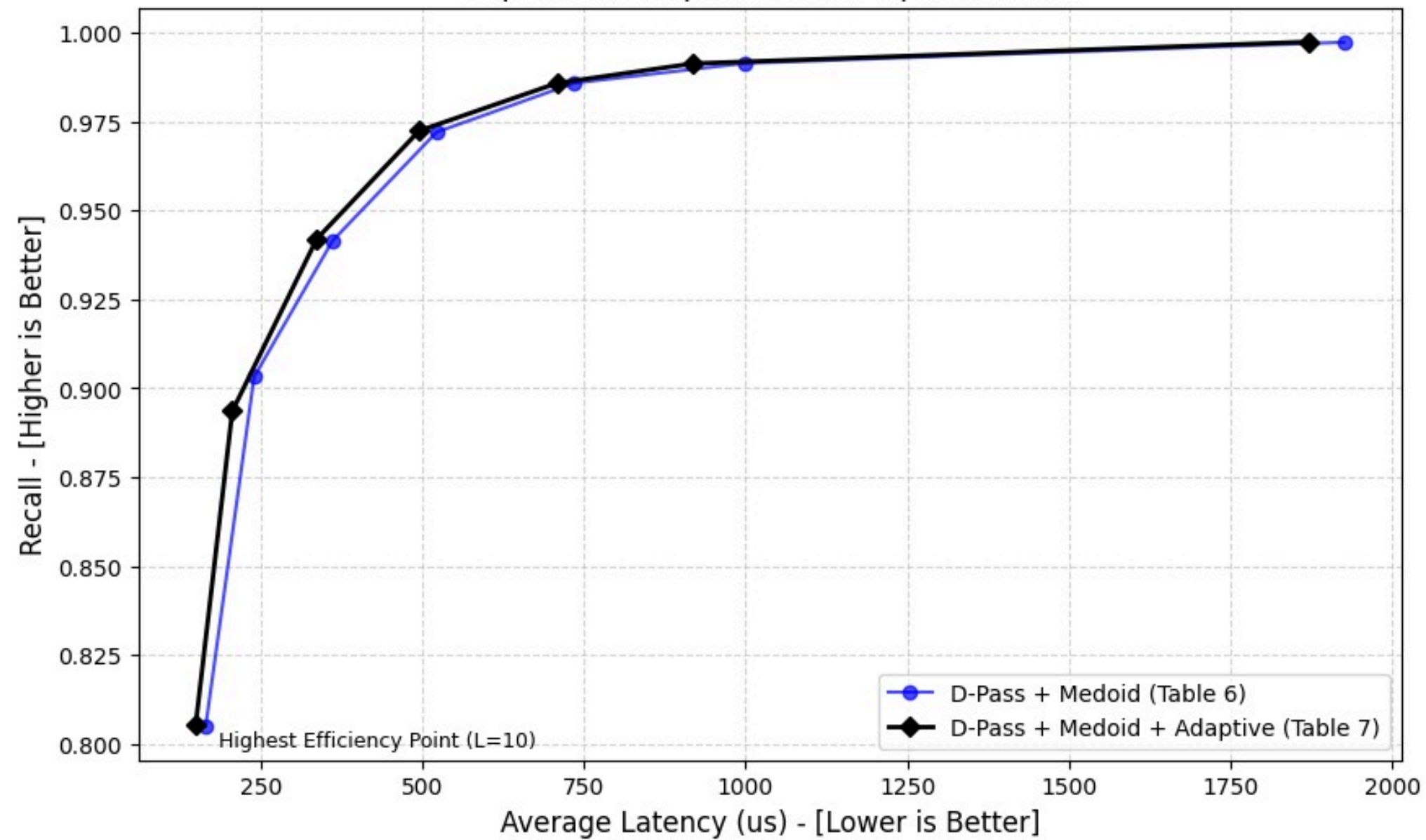
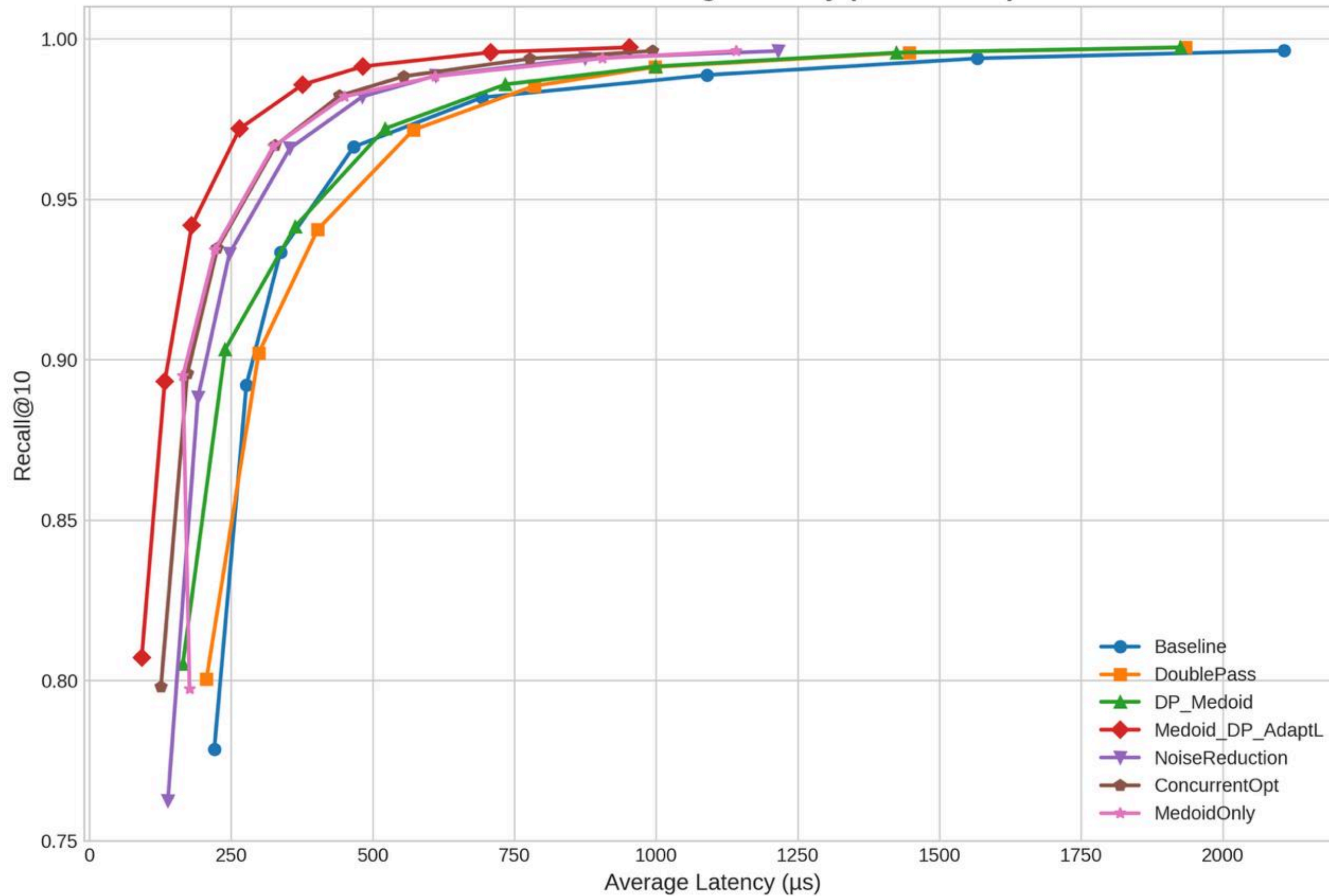


Table 2: Improvement: (Medoid + Double Pass) vs. (Medoid + Double Pass + Adaptive L)

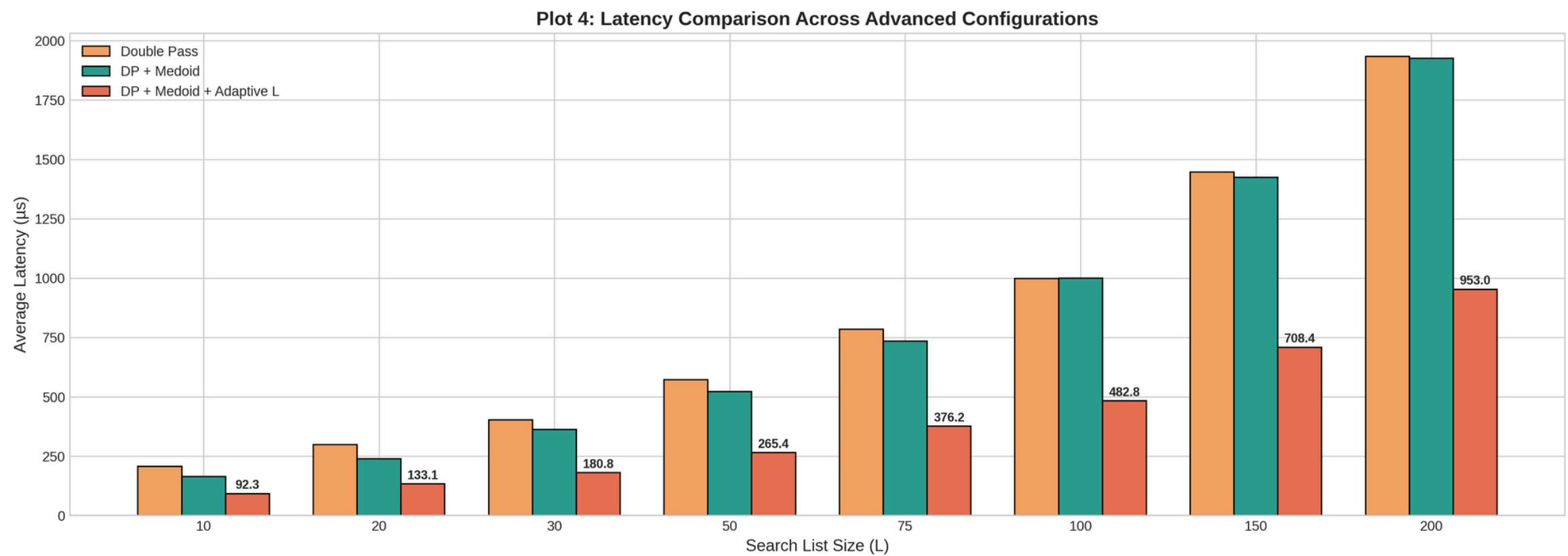
Metric	L=50 (Old → New)	L=100 (Old → New)	L=200 (Old → New)
Avg Latency	521.8 → 265.4 (+49.1%)	999.2 → 482.8 (+51.7%)	1925.0 → 953.0 (+50.5%)
Dist Cmps	1357.6 → 1379.0 (-1.5%)	2274.5 → 2287.6 (-0.5%)	3874.2 → 3882.2 (~0%)
Recall@10	0.9720 → 0.9720	0.9914 → 0.9914	0.9973 → 0.9973

RECALL vs AVERAGE LATENCY

Plot 1: Recall vs. Average Latency (All Methods)



LATENCY ACROSS ‘L’ VALUES



L Value	Double Pass (Lat)	DP + Medoid (Lat)	DP + Medoid + Adaptive L (Lat)
50	571.7 μs	521.8 μs	265.4 μs (-50%)
100	997.8 μs	999.2 μs	482.8 μs (-50%)
200	1934.2 μs	1925.0 μs	953.0 μs (-50%)

KEY CONCLUSIONS:

1. Medoid Initialization lowers latency slightly by reducing Distance Computations (better starting point).
2. Adaptive L serves as a massive multiplier, dropping latency by ~50% across all ranges without sacrificing recall.
3. At L=200, Adaptive L completes in 953μs compared to Double Pass's 1934μs, proving algorithmic heuristics scale incredibly well.

HOW WE USED AI

1

Paper Analysis

Used AI to parse and summarize the original DiskANN paper, extracting core algorithmic logics and areas of possible improvements.

3

Debugging

AI guided the `__builtin_prefetch` placement and atomic spinlock replacement with correctness reasoning.

2

Understanding implementation

AI helped navigate the GitHub codebase — identifying the single-pass construction bottleneck and memory access patterns.

4

Results Analysis

Fed it the performance tables and asked it to make plots for better visualisation and observations.

WHERE **AI** FELT SHORT

- Couldn't suggest novel improvements
- The ideas has to come from ourselves
- It couldn't reason about the specific bottlenecks of our implementation

Key Takeaway: AI is a strong executor but a weak innovator.

THANK YOU